LABORATORIO DE CONTROL, Práctica 3: Simulink y control con MATLAB

Programa de Ingeniería Electrónica
Facultad de Ingeniería
Universidad de Antioquia
2026-1

Profesor: Alumno: Grupo Lab: Fecha

1. **RESULTADOS DE APRENDIZAJE**

1.1 Conocer, con base en ejemplos, los procedimientos más comunes de la simulación de los SC con el uso de **S**IMU**L**INK, SL, la herramienta gráfica de Matlab

1.2 Aprender la forma de conectar los espacios de trabajo de **M**AT**L**AB: ML y SL.

1.3 Obtener en SL la respuesta transitoria de SC lineales (y no lineales)

2. **ACTIVIDAD DE APRENDIZAJE.**

En esta práctica se continúa con el conocimiento y uso de herramientas de análisis y diseño de Sistemas de Control, en esta oportunidad con el uso de ejemplos que permiten apreciar las bondades de la herramienta de estudio. Para agilizar el desarrollo de la práctica, se solicita al alumno previamente entrar al programa Simulink y conocer loe recursos de los que se dispone, sin pensar en llegar a un dominio de esos recursos. Para orientar al alumno en este objetivo, se le pide leer la sesión 3 de esta guía y presentar un breve preinforme

PRIMERA PARTE DEL LABORATORIO.

Como primera parte del laboratorio se requiere: documento **con las respuestas a los interrogantes. También puede ser documentado en COLAB, o documento en WORD Editable el cual se debe presentar al inicio de la sesión y enviar al CLASSROOM y correo del docente una vez finalizada la sesión**. El estudiante debe:

- Hacer una descripción somera de la función de los comandos de ML: *lsim*, *sim*, *linmod, trim*, *simset*, *simget,* y de los bloques de SL: *sign, in, out, to workspace, transfer fcn,* [1]
- Resolver los numerales 4.1 y 4.4 de esta guía.
- Además, previo al inicio de la sesión se debe estudiar el programa de la sección 4.8 y corregir cualquier error de programación que encuentre, si existe el error.
- **Se sugiere** estudiar la sección 5.0

Para el resto de la práctica, el estudiante debe tener el documento editable para continuar resolviendo los apartados del laboratorio

Modalidad: individual

3. Introducción

No se pretende con esta práctica describir completamente los procederes y metodologías que SL ofrece para la simulación y diseño de sistemas de cualquier tipo o naturaleza. Con la ayuda del profesor, quien aclarará aspectos más profundos del programa, se introduce al estudiante en el manejo de SL con base en problemas prácticos y se lo motiva a tomar iniciativa para, con el uso las ayudas que trae el programa, buscar la solución de las dificultades normales que se encontrarán. Se pide al estudiante que lea la documentación básica que trae SL, a la cual se accede desde el entorno de ML al entrar el comando **helpdesk** en la ***V****entana de* ***C****omandos de* **ML**, VCML, y siguiendo el enlace *Simulink* de la ventana *Help,* o simplemente abriendo *Simulink* con ayuda del enlace gráfico que aparece en el menú de ML y luego, en la ventana de *Simulink,* con el menú de ayuda que allí aparece. *Simulink* también se abre tecleando el comando *simulink* en la VCML.

SL es el entorno gráfico de simulación de ML para modelar, analizar y simular sistemas dinámicos, lineales o no, continuos, discretos o híbridos, con frecuencia única o múltiple de muestreo. El programa resuelve, entonces, sistemas descritos en ecuaciones diferenciales o diferencia, lineales o no, al igual que aquellos descritos por expresiones lingüísticas, como los sistemas difusos. Lo de programa gráfico se comprende con solo estudiar la figura 1, en la cual se muestra el primer programa ejemplo que se montará en esta práctica.

Cuando se escribe un programa en SL y se guarda con un nombre, por ejemplo "ejemplo1", se genera un archivo de formato especial, llamado el *archivo del modelo,* con extensión **.slx**, el cual contiene, en forma estructurada, en formato ASCII, todo lo relacionado con la parte gráfica del programa, lo que permite volverlo a reproducir cada vez que se invoca, así como las propiedades de los bloques del programa, relacionadas con los procederes matemáticos que usa el programa escrito.

Se pueden extender las posibilidades de SL mediante el uso de las llamadas *Funciones S*, FS, o funciones de sistema, que permiten *agregar a los modelos bloques propios creados en ML, C, C++, Fortran o Ada*. Como se ve, una FS es la descripción en un lenguaje de computador de un bloque de SL. SL proporciona un bloque llamado FS para acceder a la FS del usuario, escrita de acuerdo a reglas sencillas, la cual permite resolver problemas muy complejos, al poder interactuar con los algoritmos de solución de ecuaciones, continuas o discretas, que usa SL. La FS se puede usar con el toolbox *Real-Time Workshop* para generar el código C o Ada del programa SL que se desee.

4. Procedimiento

4.1 Para iniciar SL, seleccione el icono de Simulink Library en el menú gráfico de ML o escriba en la VCML,
>> simulink

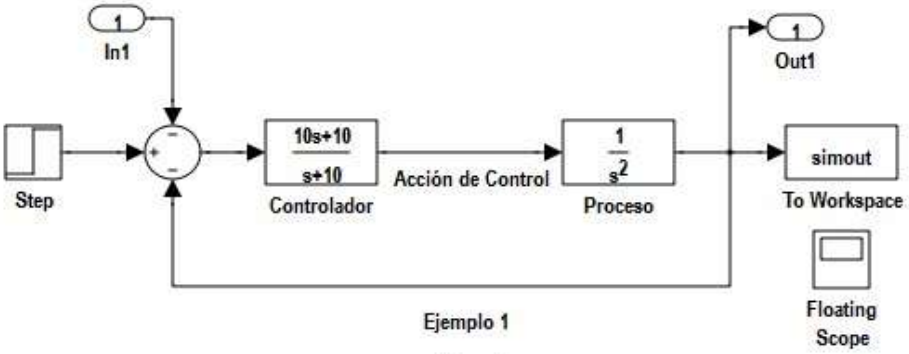
Aparece la Ventana de *Simulink* con tres pestañas: New, Examples y Learn; la pestaña New está activa por defecto. Allí se da click en la opción Blank Model, y se abre la ventana de trabajo de SL, para nuevos programas, con múltiples pestañas con menús adecuados para el estudio y diseño de sistemas. En la pestaña Simulation se observa la opción Library Browser, en la cual se hace click para conocer los recursos gráficos de los que se dispone, agrupados en sublibrerías, con iconos especializados para la simulación de sistemas, entre las que se destacan

las denominadas *Simulink, Control System* toolbox y *Simulink Extras*. Haga un rápido recorrido por las bibliotecas, para que tome una idea de los recursos que podrá usar cuando tenga necesidad de ellos.

¿Cuáles sublibrerías poseen las librerías Simulink, Control Systems toolbox y Simulink Extras. Al expandir la sublibrería Simulink se encuentran las opciones: Common used blocks, Continous, Sinks y Sources, que tienen la mayoría de los bloques que se necesitan en esta práctica. Esos bloques funcionales se parametrizan haciendo doble click sobre el bloque deseado; un click con el botón derecho del ratón sobre un bloque, permite abrir una ayuda descriptiva del bloque.

4.2 En el menú gráfico de la librería a usar, haga doble click sobre el icono de creación de un nuevo modelo y arrástrelo hasta la ventana Untitle que aparece con la pestaña Simulation activa; **monte el programa de la figura 1**; el *scope* flotante lo encuentra tecleando *floating scope* en el espacio de búsqueda Library Browser. Llame y guarde el proyecto como "ejemplo1". Observe que se tiene un Sistema de Control Retroalimentado, SCR, cuyo proceso es un doble integrador (use 2 bloques integradores para el montaje) y cuyo controlador es un subsistema llamado compensador de adelanto o controlador PD real. Con este programa se aprenderá a simular de dos formas; primero, directamente desde SL (secciones 4.3 a 4.5), para lo cual no requiere los puertos *In* y *Out* del diagrama (se pueden eliminar); segundo, desde la VCML (secciones 4.6 a 4.10), mediante el uso de los puertos *In* y *Out*, para lo cual no es necesario el bloque de señal escalón

4.3 Haga doble click sobre el icono del *scope* flotante y observe el menú de opciones que aparece en la ventana *floating scope*. Toque con el puntero del ratón los iconos del menú para que se entere de su función.
Haga click sobre el menú *View>> Configuration properties* y aparecerá una ventana con cuatro lengüetas que permiten definir, seleccionar y usar características especiales del bloque. Con la lengüeta Logging se accede a la opción de enviar los datos de la gráfica al ***E**spacio de **T**rabajo de **ML***, ETML, o *workspace,* con un nombre de variable seleccionable y un formato de variable específico: *structure with time*, *structure* o *array*. Por ahora use el Scope normal (no flotante) y agregue al programa de la figura 1 un bloque *mux*, cuya salida se conecta a la entrada del *scope* y cuyas entradas se conectan con las señales de entrada, salida y de acción de control del sistema a estudiar (tres entradas en el bloque *mux)* . Cuando más adelante corra el programa, notará qué pasa con los iconos restantes del menú del *scope*. Se puede tener más información si en la VCML se escribe el comando: >>doc scope. El comando *doc* se puede usar para obtener ayuda sobre la función de cada uno de los iconos de SL



Ejemplo 1

Figura 1

4.4 Haga doble click sobre los iconos *In1, Out1 y To Workspace* (para el preinforme lo puede hacer en la librería de SL directamente, sin que necesite montar el programa de la figura 1) y estudie las posibilidades que ofrece cada uno de ellos.

¿Qué diferencia de funcionalidad existe entre *Out* y *to Workspace*?

Observe que en el programa SL se tienen varias maneras de enviar información de la simulación al ETML.

4.5 Para simular o correr el programa, se lo debe *parametrizar*, es decir definir el entorno de simulación. Siguiendo la lengüeta *Modeling* de la ventana del programa, escoja la opción *Model Settings>>Model settings*, y estudie las diferentes opciones que ofrece la ventana que se abre, especialmente las de las opciones *Solver, Data Import/export y Diagnostics*. En *Solver* se puede seleccionar el método de solución (solver), de paso variable o fijo, de la ecuación que rige el programa gráfico que se monta en SL, el tiempo de simulación y otros conceptos asociados a los métodos numéricos de solución de ecuaciones diferenciales; en *Data Import/export* se decide si se reciben datos desde el ETML, en este caso con el uso del puerto *In1*, o si los resultados se pasan al ETML (cuáles datos se pasan, con cuál formato se pasan y cuánto se pasa de cada dato) con uso del puerto *out1*; y en *Diagnostics*, las condiciones de chequeo que deben observarse al correr el programa (alarmas y paro del programa, si ocurre algún problema durante la simulación). En *Data Import/export,* seleccione guardar las variables de simulación *time*, *states* y *output* en el ETML en *formato* array. Puede dejar los nombres por defecto de las variables. El profesor le ayudará al respecto.

Ahora ya puede simular, pero antes de hacerlo verifique que no haya variables definidas en el ETML. Para ello, en la VCML, escriba: >> whos. Si aparece cualquier variable bórrela (no use el comando *clear all* porque borrará variables del inicio de trabajo en la práctica) y así se tendrán sólo las variables que generará el programa en SL cuando se corra. También puede borrar las variables en la ventana ETML (workspace)
Vaya de nuevo a SL, active el *scope* y observe las señales almacenadas. Haga un esquema de esas señales

En el ETML se tienen, ahora, varias variables generadas como resultado de la corrida del programa en SL. ¿Cuáles variables aparecen en el ETML? Asócielas con la parametrización que haya hecho del programa en SL y/o los bloques del programa.

¿Tiene alguna variable no esperada?

Con la información contenida en las variables que tiene en el ETML, grafique la respuesta del sistema con el uso del comando plot. Recuerde que las variables están en formato arreglo (con tiempo) y que para poder graficarlas primero debe extraer la información de los arreglos ¿Cómo se hace esto último? (¿Cómo extrae usted una columna de una matriz). También puede usar indexación sobre cada columna

4.6 Para iniciar la simulación de un programa SL desde el ETML sin abrir SL, es conveniente que se estudien y comprendan los comandos *lsim* y *sim* de ML.
¿Qué diferencia importante se da entre los dos comandos?

¿Por qué son importantes en el modelo SL los iconos *in port* y *out port*?

4.7 Antes de correr el programa 'ejemplo1', en la VCML entre los siguientes comandos (también puede usar el editor de comandos, invocándolo con el mando *edit*)

```
>> whos   % Para conocer las variables que actualmente tiene en el ETML; también las puede ver en la ventana
% workspace de ML
>> clear (nombres de variables a eliminar);      % Elimine todas las variables del paso anterior
>> whos       % Para verificar el borrado de variables; también lo puede ver en la ventana workspace de ML
>> out = sim('ejemplo1', 'StopTime', '5') % Se llama al programa en SL 'ejemplo1'
 >> whos                          % Se conocen las variables creadas en el paso anterior
```

¿Cuántas variables tiene ahora el ETML?
¿De qué tipo?

¿Tienen alguna relación?

¿Cómo cambia el número de variables y su tipo si se ejecuta el comando >> out = sim('ejemplo1')?

4.8 Al sistema de la figura 1 agregue un bloque de saturación, con valores de saturación +1 y 1, para obtener el sistema **no lineal** de la figura 2. Se quiere estudiar con este sistema el efecto de la saturación en la respuesta temporal del mismo, especialmente los cambios que puedan existir en el Sobre nivel Porcentual, SP, y el tiempo de estabilización, ts, cuando el valor de la señal de entrada cambia entre 1.5 y 7.5 en incrementos de una unidad. (Observe que el programa en SL se debe ejecutar 7 veces y, como no es práctico hacerlo manualmente, se escribe un programa en ML que haga tal trabajo. Es en estas situaciones cuando conviene saber trabajar con la información de las variables estructuradas.

Parametrice el programa de SL de modo que tanto el Scope como la simulación envíen datos al ETML. Guarde el programa SL, con el nombre "ejemplo2".

Se debe escribir un programa en ML que permita tener todas las respuestas temporales en una sola gráfica, en una simulación de 100 puntos entre 0 y 9.9 segundos. Antes de graficar, los resultados de la simulación se interpolan para tener unas gráficas más exactas. El programa debe calcular el valor de estado estable y el SP para cada respuesta. Se usa el siguiente programa para hacer lo solicitado [2]. Nota: el siguiente programa puede contener errores que impiden que funcione y que el estudiante debe corregir.

```
% Programa que obtiene y grafica en una sola gráfica la respuesta temporal del sistema Simulink
% 'ejemplo2'
```

```
% cuando la entrada toma valores entre 1 y 7 con incrementos unitarios, para observar el efecto de un
% bloque de saturación en un sistema de control. El programa calcula solo 100 puntos de simulación, entre 0 y
% 9.9 segundos,  los cuales se completan mediante un procedimiento de interpolación para obtener unas gráficas
 % más suaves. El programa también calcula el valor de estado estable de cada respuesta y su sobre nivel
% porcentual.

 clear variable1 variable2 …     % Borre una a una todas las variables del espacio de trabajo de ML
 .                               % No use los comandos clear all o clear sin nombre de variables
 t=[0:0.1:10]' ;   % Se define el tiempo de simulación, equivalente exactamente a 100 instantes de simulación
                   % en un vector columna. No olvide la comilla simple
for i=1:7
     ut=[t,i*ones(size(t))];                % Se generan las señales tipo escalón de entrada al sistema
      [tt,xx,yy]=sim('ejemplo2',9.9,[],ut);              % Se corre el programa Simulink y los resultados se
                                                 % guardan en las variables tt,xx,yy.
      yint=interp1(tt,yy,t-0.5);   % La interpolación crea puntos adicionales para unos resultados más precisos
      y0(1:size(yint),i)=yint;          % Se generan las siete respuestas del sistema
       i=i+1;
end
sy=size(y0);                                % Se halla la dimensión de la variable
y0ss=y0(sy(1),:);                           % Valor de respuesta en estado estable
SP=100*(max(y0)-y0ss)./y0ss;            % Sobre nivel porcentual. No pase por alto el punto
plot(t,y0); grid                        %Se grafica el resultado parcial y se le agrega malla a los ejes
```

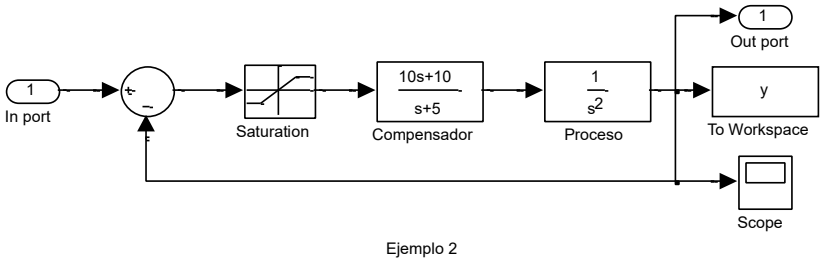


Figura 2

El anterior programa en lenguaje .m de ML se guarda en *el mismo directorio* donde se encuentra el programa SL, *pero con extensión .m*, con un nombre diferente, por ejemplo "ejemplo2m". En el VCML escriba

```
>> ejemplo2m    % Se ejecuta " ejemplo2.mdl" o "ejemplo2.slx" y obtendrá los resultados pedidos.
```

¿Cuántas respuestas tiene la variable generada por el bloque Scope? (dé una explicación)

¿Cuántas respuestas tiene la variable generada por el bloque To Workspace?

¿Cuántas respuestas tiene la variable y0?

¿Cuál es la respuesta con mayor sobre nivel porcentual?

¿Cuál es la respuesta con menor tiempo de estabilización?

Use el comando *linmod* con este programa SL ¿Para qué sirve el comando?¿Escriba lo que le devolvió ML al usar el comando

Recuerde que ML (y SL) nombra como x_1 a la variable de estado más hacia la izquierda del diagrama y como x_n a la variable de salida. Compruebe manualmente el resultado que obtiene con el comando especificado.

5.0 Conclusiones

6.0 Bibliografía

[1] Mathworks, "Simulink", Help Document in Matlab releases 2013, 2016, 2020, 2022

[2] Shashian, B. and Hassul, M. "Control Systems Design Using Matlab", Prentice Hall, 1993, p. 154